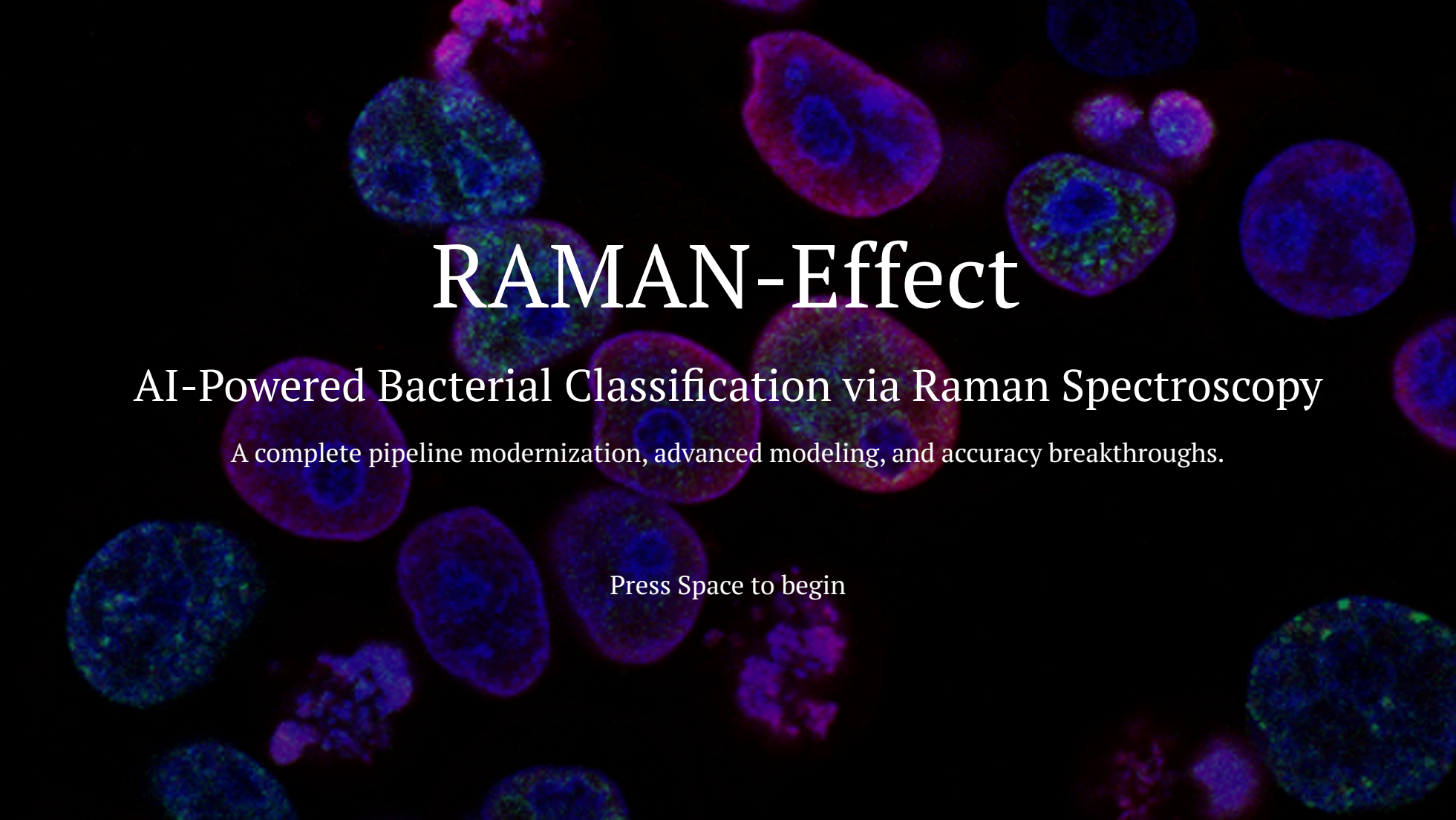


A microscopic image of several bacterial cells. The cells are irregularly shaped and appear to be stained with two different fluorescent dyes, one red and one green. The red dye highlights the cell walls or membranes, while the green dye highlights internal structures, possibly the nuclei or specific organelles. The background is dark, making the fluorescent cells stand out.

# RAMAN-Effect

## AI-Powered Bacterial Classification via Raman Spectroscopy

A complete pipeline modernization, advanced modeling, and accuracy breakthroughs.

Press Space to begin

# 1. Pipeline Modernization

We migrated from a monolithic, legacy structure to a **modular, production-ready PyTorch architecture**.

- **Modular Files:** Split the giant script into `dataset.py` , `model.py` , `engine.py` , and `main.py` .
- **Clean Root:** Organized all scripts into `src/` , notebooks into `notebooks/` , and tests into `tests/` .
- **CLI Tooling:** Built a powerful `main.py` entrypoint equipped with `argparse` for easy hyperparameter tuning.
- **Code Quality:** Applied formatting to optimize imports and scrubbed git history of compiled artifacts.

## 2. Dataset Overview and Preprocessing

Bridging the gap between Complex Chemistry and Data Science.

- **Dataset Specs:** 60,000 perfectly balanced samples across 30 bacterial classes.
- **Features:** 1,000 points per spectrum, corresponding to wavenumbers ranging from 381.98 to 1792.4  $\text{cm}^{-1}$ .
- **Physical Meaning:** Raman spectroscopy acts as a chemical fingerprint scanner. The data represents intensity shifts caused by molecular vibrations.

### RamanSPy Preprocessing Pipeline

1. **Whitaker-Hayes Despiking:** Removes random high-energy cosmic ray artifacts.
2. **Savitzky-Golay Denoising:** Smooths high-frequency thermal and sensor noise while preserving peak shapes.
3. **ASPLS Baseline Correction:** Flattens curved background auto-fluorescence typical in biological samples.
4. **Min-Max Normalization:** Scales spectra between 0 and 1 for consistent neural network input.

# 3. Model Architecture

## The Legacy Baseline

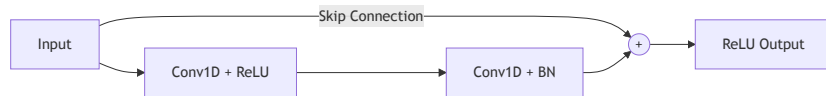
We ported the original 6-layer CNN directly into PyTorch to establish a fair baseline.

- Shallow Architecture
- Baseline Validation Accuracy: **78.70%**
- Prone to the vanishing gradient problem if scaled deeper.

## The New Brain: ResNet

A deeper 1D Residual Network utilizing Skip Connections.

- **Solves Vanishing Gradients:** Skip connections bypass standard convolutions, allowing deeper layers to learn.
- **Preserves Raw Signal:** Subtle Raman peaks are carried forward directly to deeper abstract layers.



## 4. The Overfitting Discovery

In our early tests, the raw power of the ResNet became apparent, but it came with a catch.

- **Massive Capacity:** The ResNet easily memorized the dataset, hitting **99.30% Training Accuracy**.
- **The Catch:** Validation accuracy actually dropped to **76.67%**.
- **Diagnosis:** The model was so powerful it began memorizing the exact spectrometer noise and minor intensity variations of the training samples instead of learning the underlying chemical fingerprints.

## 5. Data Augmentation

To force the model to generalize to real chemistry rather than memorizing noise, we applied physics-aware data augmentations.

$$\mathbf{x}_{aug} = \left( \text{Roll}(\mathbf{x}_{raw}, \Delta_{shift}) + \mathcal{N}(0, \sigma^2) \right) \times \mathcal{U}(0.95, 1.05)$$

- **Random Shift (Roll):** Simulates instrument calibration drift by shifting the spectrum left or right by up to 5 indices.
- **Gaussian Noise:** Injects random noise to simulate thermal or sensor variance from the detector.
- **Random Scale:** Scales intensity by  $\pm 5\%$  to simulate variations in laser power or sample concentration.
- **Regularization:** We also increased Dropout from 0.2 to 0.4.

# Implementation: AugmentedDataset

Here is exactly how we implemented the augmentation inside our PyTorch Dataset.

```
class AugmentedDataset(Dataset):
    def __init__(self, dataset, augment=False):
        self.dataset = dataset
        self.augment = augment

    def __getitem__(self, idx):
        x, y = self.dataset[idx]

        if self.augment:
            x_np = x.numpy()

            # 1. Random shift (roll)
            shift = np.random.randint(-5, 6)
            x_np = np.roll(x_np, shift, axis=-1)

            # 2. Random Gaussian noise
            noise = np.random.normal(0, 0.01, x_np.shape).astype(np.float32)
            x_np = x_np + noise

            # 3. Random scale
            scale = np.random.uniform(0.95, 1.05)
            x_np = x_np * scale
```

## 6. Training & Optimization Strategy

Beyond architecture and augmentation, achieving high accuracy required robust training mechanics to prevent the model from memorizing noise.

- **Weight Decay (L2 Regularization):** Added a penalty (0.01) to the Adam optimizer to keep network weights small, preventing the model from hyper-focusing on random noise spikes.
- **Cosine Annealing Scheduler:** Smoothly decreases the learning rate following a cosine curve, allowing the model to make fine adjustments as it approaches the optimal solution.
- **Two-Phase Training Strategy:**
  1. **Pre-training:** Train on 60,000 lab reference samples to learn the foundational chemical signatures.
  2. **Fine-tuning:** Drop the learning rate by 10x and train on a smaller fine-tuning dataset to adapt to specific clinical variations without forgetting the foundations.

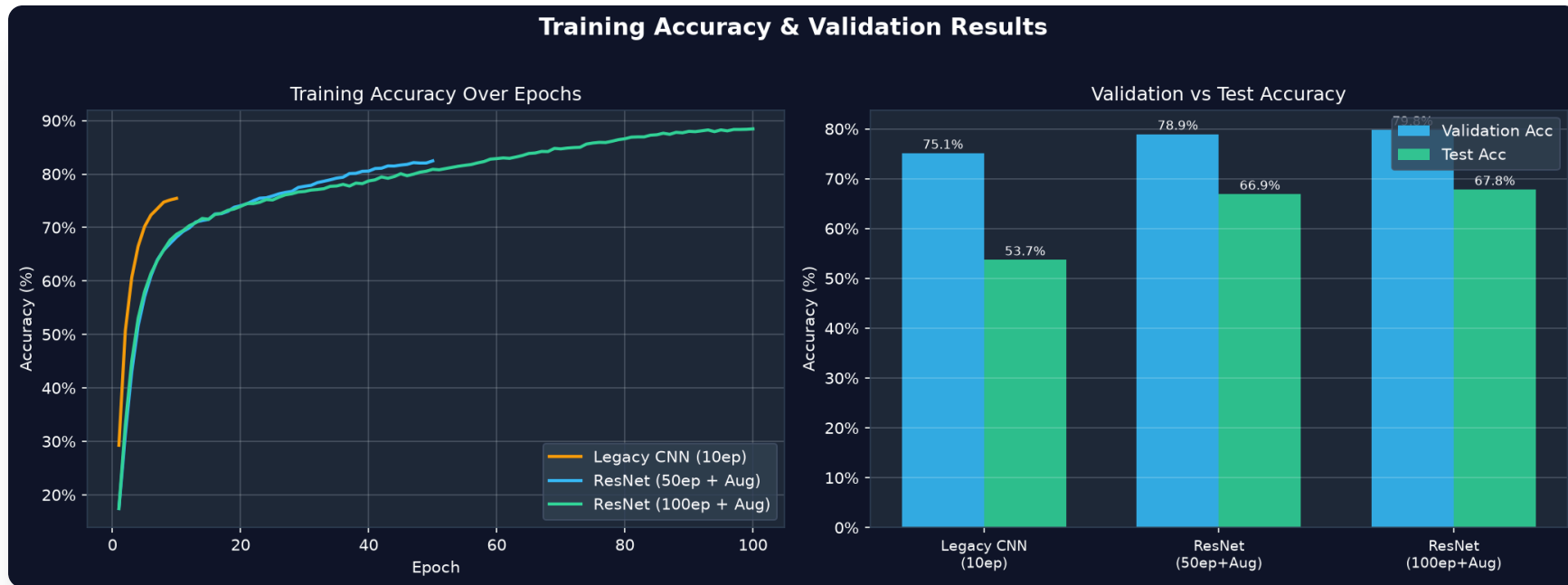
## 7. Final Results: Defeating the Baseline

By forcing the ResNet to learn general chemistry rather than noise, we achieved a massive victory.

| Metric                | Legacy CNN | ResNet (10ep) | ResNet (50ep + Aug) | ResNet (100ep + Aug) |
|-----------------------|------------|---------------|---------------------|----------------------|
| <b>Training Acc</b>   | 79.28%     | 80.41%        | 82.46%              | <b>88.44%</b>        |
| <b>Validation Acc</b> | 78.70%     | 77.26%        | 78.93%              | <b>79.80%</b>        |
| <b>Test Acc</b>       | 53.70%     | 59.47%        | 66.90%              | <b>67.80%</b>        |

**The Ultimate Proof:** When finally evaluated on the completely unseen `x_test.npy` hold-out data, the augmented ResNet jumped to **67.80% Accuracy** (crushing the baseline's 53.70% score by +14.1%).

## 7b. Results: Training Curves



## 8. Future Work

What is next for RAMAN-Effect?

- **MixUp Augmentation:** Simulating mixed bacterial cultures mathematically.
- **1D Transformers:** Using Attention mechanisms to correlate distant chemical peaks.
- **SHAP Interpretability:** Generating heatmaps to prove to chemists exactly which biological markers the model is looking at.

# Thank You!

The RAMAN-Effect pipeline is now modular, physically-aware, and mathematically robust.